



Pensumhefte, 2026

Algoritmer og datastrukturer

Innledning

Dette pensumheftet er en kombinasjon av tre ting: (i) En forelesningsplan; (ii) en pensumliste; og (iii) et mini-kompendium. For hver forelesning er det listet opp hva som er hovedtema og hvilken del av pensum forelesningen bygger på. I tillegg har jeg laget en detaljert opplisting av læringsmålene i emnet, for å tydeliggjøre hva det er meningen at dere skal kunne. Bakerst i heftet er det noen appendikser med ekstra materiale, og det utgjør mini-kompendie-delen. Sammen med pensumboka, forelesningene og øvingene, håper jeg at dette gir dere det dere trenger for å sette dere inn i stoffet, men det er viktig å huske på at *det er dere som må gjøre jobben med å lære det!*

Her er det ikke bare faktakunnskap man skal tilegne seg; det er ideer som krever modning over litt tid, og det er derfor viktig at dere **begynner å jobbe tidlig**, og at dere **jobber jevnt**.

Det er ikke så taktisk å utsette pensumlesningen til eksamensperioden, og håpe at et skipertak skal ordne biffen.

Det er også viktig å merke seg at læreboka [2] er viktig i dette emnet.* Det er flere måter å fremstille algorit mestoff på, og man kan selvfølgelig bruke de ressursene man vil for å lære seg det. Det finnes mange videoforelesninger og animasjoner og lignende på nett, og mye er forklart ganske rett frem i Wikipedia og ellers. For å få skreddersydde forklaringer, er også KI-verktøy nyttige, selv om det der er spesielt viktig at dere sjekker at forklaringene dere får er riktige! Men til syvende og sist så skal det jo være en eksamen i emnet, og der må vi ha en eller annen form for standardisering, av rent praktiske hensyn.

Derfor baserer vi oss på antagelsen om at **dere faktisk leser læreboka**, og at dere leser jevnt gjennom semesteret.

En mulig fremgangsmåte kan være:

1. Kikk igjennom relevant pensum før forelesningen, uten å lese grundig;
2. Se forelesningen, og delta i diskusjon/refleksjon; og så
3. Les pensumet grundigere, med forklaringene fra forelesningen i ryggen.

Å skumme igjennom stoffet på forhånd vil forhåpentligvis gjøre at du begynner å lure på enkelte ting, som igjen gjør at du kan få mer ut av forelesningen. Det vil også gi deg et visst overblikk. Forelesningene vil så gi deg et første ordentlig møte med stoffet, slik at det forhåpentligvis blir lettere å forstå boka når du setter deg ned for å lese selv. Det er ingen grunn til panikk om du ikke skjønner absolutt alt i en forelesning, eller om du ikke husker alt etterpå. Sannsynligvis er du bedre rustet til å jobbe med stoffet uansett. Når du så har begynt å lese stoffet, kan du også jobbe med den tilhørende øvingen. Ta gjerne notater og gjør [skriftlig refleksjon](#) som en del av studiearbeidet.

Et hovedtips til studeringen kan være:

* Det er en del endringer i denne utgaven, så man bør bruke den, og ikke den forrige [1].

Test deg selv!

Det er et velkjent fenomen fra f.eks. forskning på hukommelse at om man leser en tekst flere ganger så kan man *lures til å tro at man husker den*, og kanskje til og med *forstår den*, bare fordi den virker kjent. Dette er en skummel felle å gå i. Det er lett å tro at man har skjønt mer enn man har! Øvingene gir deg en god mulighet til å teste hva du faktisk har fått med deg, men det går jo også an å finne andre måter å gjøre det på.

Ikke bare er det å gjøre oppgaver en viktig måte å teste deg selv på; det er også en måte å styrke hukommelsen på, såkalt *recall training*. Det er mulig stoffet ligger godt lagret, men om du ikke trener på å hente det frem, så kan det være du ikke husker det likevel. Dessuten er oppgaver en god trening i å anvende ferdighetene du tilegner deg i emnet. For hver øving, har vi oppgitt en liste med anbefalte oppgaver, ut over dem som faktisk inngår i øvingen. I øvingstimene vil det også være mulighet til litt interaktiv oppgaveløsning, for dem som har behov for mer. Om du er interessert, så finnes det også store mengder med algoritmeløsningsoppgaver på nett, både knyttet til programmeringskonkurranser og til jobbintervjuer.

En siste, kanskje litt pussig gevinst ved å løse oppgaver er at det kan bidra til å bygge opp mentale strukturer som er *mottakelige for relaterte ideer*. Dette er det såkalte *pretesting*-fenomenet: Om du prøver deg på en oppgave fra en del av pensum du ikke har lært noe om ennå, selv om du *vet* at du ikke kommer til å få den til, så kan det bidra til at du lærer stoffet bedre når du begynner å jobbe med det. Du trenger ikke se på løsningen til oppgaven en gang. Her er det mange muligheter. Det er for eksempel fullt av oppgaver i læreboka, og dem kan det være lurt å kikke igjennom *før* du begynner å lese; kanskje til og med *før* forelesningen!

Dette gjelder også gamle eksamensoppgaver, naturligvis. Vi prøver å variere form og innhold, så man ikke skal kunne «pugge eksamen», men det kan likevel være lurt å øve seg på gamle eksamener. De nyeste vil trolig være mest relevante. Her også kan pretesting være en tanke.

Hva med å prøve deg på et par eksamenssett allerede den første uka?

Og skulle det være noe du ikke forstår, noe du trenger hjelp til eller noe du vil slå av en prat om, så ta gjerne kontakt! Det enkleste er kanskje å oppsøke en studass, fysisk eller digitalt. Om det er spørsmål du tror flere kan ha nytte av å se svaret på, så spør svært gjerne i [emnets forum](#) på nett. Ellers går det fint an å sende e-post (for eksempel til algdatt@idi.ntnu.no) eller ta personlig kontakt med øvingsstaben eller meg.

Trenger du studieveiledning, tilrettelegging, eller bare noen å snakke med? Se også oversiktssiden om veiledning for studenter: i.ntnu.no/veiledning

Om det er uvant å skulle lære seg relativt teoretiske ting med høy grad av presisjon, så finnes det generelle råd og tips rundt dette. En bok jeg har anbefalt flere, og fått gode tilbakemeldinger på, er denne:

Bokanbefaling

A mind for numbers av Barbara Oakley [4]. Nært knyttet til boka er *Coursera*-kurset *Learning how to learn: Powerful tools to help you master tough subjects*.

Overordnede læringsmål

De overordnede læringsmålene for emnet er som følger.*

Dere skal ha kunnskap om:

- [X₁] Et bredt spekter av etablerte algoritmer og datastrukturer
- [X₂] Klassiske algoritmiske problemer med kjente effektive løsninger
- [X₃] Komplekse problemer uten kjente effektive løsninger

Dere skal kunne:

- [X₄] Analysere algoritmers korrekthet og effektivitet
- [X₅] Formulere problemer så de kan løses av algoritmer
- ! [X₆] Konstruere nye effektive algoritmer

Dere skal være i stand til:

- [X₇] Å bruke eksisterende algoritmer og programvare på nye problemer
- [X₈] Å utvikle nye løsninger på praktiske algoritmiske problemstillinger

Mer spesifikke læringsmål er oppgitt for hver forelesning.

I listene med læringsmål er noen merket med *utropstegn*. Dette er en ganske uformell greie, og ikke noe å legge for stor vekt på. Det er men er ment å indikere temaer som kanskje er viktigere *enn man skulle tro*.

Av og til betyr det at de merkede målene er viktigere enn de andre, men for mer marginale ting, så er det mer «Obs! Dette bør du ikke nedprioritere!»

Punkter nedenfor merket med □ er pensum. Kapittelreferanser er til pensumboka *Introduction to Algorithms* [2].

* Se emnebeskrivelsen, <https://ntnu.no/studier/emner/TDT4120>.

Forkunnskaper

En viss bakgrunn er nødvendig for å få fullt utbytte av emnet, som beskrevet i emnets anbefalte forkunnskaper.* Mye av dette kan man tilegne seg på egen hånd om man ikke har hatt relevante emner, men det kan da være lurt å gjøre det *så tidlig som mulig i semesteret*.

Dere bør:

- [Y₁] Kjenne til begreper rundt *monotone funksjoner*
- [Y₂] Kjenne til notasjonene $\lceil x \rceil$, $\lfloor x \rfloor$, $n!$ og a mod n
- [Y₃] Vite hva *polynomer* er
- [Y₄] Kjenne grunnleggende potensregning
- [Y₅] Ha noe kunnskap om grenseverdier
- ! [Y₆] Være godt kjent med logaritmer med ulike grunntall
- [Y₇] Kjenne enkel sannsynlighetsregning, indikatorvariable og forventning
- [Y₈] Ha noe kjennskap til rekkesummer (se også side 17 i dette heftet)
- [Y₉] Beherske helt grunnleggende mengdelære
- [Y₁₀] Kjenne grunnleggende terminologi for relasjoner, ordninger og funksjoner
- [Y₁₁] Kjenne grunnleggende terminologi for egenskaper ved grafer og trær
- [Y₁₂] Kjenne til enkel kombinatorikk, som permutasjoner og kombinasjoner

Vi forventer også at dere behersker grunnleggende programmering, at dere har noe erfaring med rekursjon og grunnleggende strukturer som tabeller (*arrays*) og lenkede lister. Dere vil også få noe innføring i dette i forelesning 2a med tilhørende øving.

De følgende punktene er ment å dekke de anbefalte forkunnskapene. De er i prinsippet pensum, men antas i hovedsak kjent (bortsett fra bruken av asymptotisk notasjon). De vil derfor i liten grad dekkes av forelesningene. De er også i hovedsak ment som bakgrunnsstoff, til hjelp for å tilegne seg resten av stoffet, og vil dermed i mindre grad bli spurt om direkte på eksamen.

- ☐ Kap. 3. Characterizing running times: 3.3
- ☐ Kap. 5. Probabilistic analysis . . . : s. 130–131
- ☐ App. A. Summations: s. 1140–1141 tom. ligning (A.3) samt ligning (A.6)
- ☐ App. B. Sets, etc.
- ☐ App. C. Counting and probability: s. 1178–1180, C.2 og C.3 tom. ligning (C.27)

Vi vil generelt prøve å friske opp relevant kunnskap, men om dette stoffet er ukjent, så regnes det altså stort sett som selvstudium. Dersom det er noe dere finner spesielt vanskelig, så ta kontakt.

* Se emnebeskrivelsen, <https://ntnu.no/studier/emner/TDT4120>.

Gjennom semesteret

Det følgende er pensum knyttet til flere forelesninger gjennom semesteret:

- ☐ Lysark fra ordinære forelesninger
- ☐ Appendiks til dette dokumentet

Læringsmål for hver algoritme:

- [Z₁] Kjenne den formelle definisjonen av det generelle problemet den løser
- [Z₂] Kjenne til eventuelle tilleggskrav den stiller for å være korrekt
- [Z₃] Vite hvordan den oppfører seg; kunne utføre algoritmen, trinn for trinn
- ! [Z₄] Forstå korrekthetsbeviset; hvordan og hvorfor virker algoritmen egentlig?
- [Z₅] Kjenne til eventuelle styrker eller svakheter, sammenlignet med andre
- [Z₆] Kjenne kjøretidene under ulike omstendigheter, og forstå utregningen

Læringsmål for hver datastruktur:

- [Z₇] Forstå algoritmene (jf. mål Z₀₁–Z₀₆) for de ulike operasjonene på strukturen
- [Z₈] Forstå hvordan strukturen representeres i minnet

Læringsmål for hvert problem:

- [Z₉] Kunne angi presist hva input er
- [Z₁₀] Kunne angi presist hva output er og hvilke egenskaper det må ha

Hva betyr «forstå»?

Gode læringsmål og læringsutbyttebeskrivelser formuleres gjerne som hva man skal *være i stand til* om man oppnår målet. For enkelhets skyld har jeg flere steder brukt begrepet *forstå*. Dette kan du i praksis oversette til *kunne forklare*.

I pensumheftet brukes en del norske begreper, mens læreboka er på engelsk. En oversikt over oversettelser som brukes finner du på nettsidene:

algdatt.idi.ntnu.no/ord.html

Ukens snubletråd: Under hver forelesning fremheves ett poeng som mange ser ut til å synes er litt utfordrende å få helt taket på. Bruk gjerne litt ekstra tid på å være sikker på at du forstår det!

Pensum

Delkapitler er også listet opp under tilhørende forelesning, sammen med læringsmål. Kapitelreferanser er til *Introduction to Algorithms* av Cormen mfl. [2].

- ☐ Innledningene til del I–VI
- ☐ Kap. 1. The role of algorithms in computing
- ☐ Kap. 2. Getting started
- ☐ Kap. 3. Characterizing running times: Innledning, 3.1 og 3.2
- ☐ Kap. 4. Divide-and-conquer: Innledning og 4.3–4.5
- ☐ Kap. 6. Heapsort
- ☐ Kap. 7. Quicksort
- ☐ Kap. 8. Sorting in linear time
- ☐ Kap. 9. Medians and order statistics
- ☐ Kap. 10. Elementary data structures: 10.3
- ☐ Kap. 12. Binary search trees
- ☐ Kap. 14. Dynamic programming: Innledning og 14.1, 14.3–14.4
- ☐ Kap. 15. Greedy algorithms: Innledning og 15.1–15.3
- ☐ Kap. 19. Data structures for disjoint sets: Innledning, 19.1 og 19.3
- ☐ Kap. 20. Elementary graph algorithms
- ☐ Kap. 21. Minimum spanning trees
- ☐ Kap. 22. Single-source shortest paths: Innledning og 22.1–22.3
- ☐ Kap. 23. All-pairs shortest paths: Innledning og 23.2
- ☐ Kap. 24. Maximum flow
- ☐ Kap. 25. Matching in bipartite graphs: Innledning
- ☐ Kap. 34. NP-completeness
- ☐ Appendiks A og C–G i dette heftet
- ☐ Lysark fra ordinære forelesninger

Støttelitteratur

Det følgende er *ikke* pensum, men er ment å være til hjelp i arbeidet med resten av stoffet. Delkapittel B.2 er knyttet til forelesning 13; resten hører til uke 2.

- ☐ Innledningen til del III
- ☐ Kap. 10. Elementary data structures: Innledning, 10.1 og 10.2
- ☐ Kap. 11. Hash tables: Til og med 11.3.1
- ☐ Kap. 16. Amortized analysis: Side 460–463 (tom. «at most 3»)
- ☐ Appendiks B i dette heftet

Forelesning 1

Algoritmer og kompleksitet

Mye påvirker ytelse, som maskinvare, programmeringsspråk og kodekvalitet. Vi skal se at valg av algoritme kan spille en større rolle enn alt dette, så forskjellene fort må måles på en geologisk eller kosmisk tidsskala!

Pensum:

- ☐ Innledningen til del I
- ☐ Kap. 1. The role of algorithms in computing
- ☐ Kap. 2. Getting started: Innledning, 2.1 og 2.2
- ☐ Kap. 3. Characterizing running times: Innledning, 3.1 og 3.2
- ☐ Kap. 25. Matching in bipartite graphs: Innledning

Læringsmål:

- [A₁] Forstå bokas *pseudokode*-konvensjoner
- [A₂] Kjenne egenskapene til *random-access machine*-modellen (RAM)
- [A₃] Kunne definere *problem*, *instans* og *problemstørrelse*
- ! [A₄] Kunne definere og bruke *asymptotisk notasjon*, O , Ω , Θ , o og ω
- ! [A₅] Kunne definere *best-case*, *average-case* og *worst-case*
- ! [A₆] Forstå at alle av O , Ω , Θ , o og ω kan beskrive *best-*, *worst-* og *average-case*
- ! [A₇] Forstå *løkkeinvarianter* og *induksjon*
- [A₈] Forstå INSERTION-SORT

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Mange synes til å begynne med at det er litt rart at O kan brukes om beste tilfelle, og Ω om verste, etc., som nevnt i læringsmål [A₆]. Men husk at O og Ω , etc., bare gir oss *øvre* og *nedre grenser*. Det å bruke grensen $\Omega(n)$ om den verste kjøretiden til INSERTION-SORT, for eksempel, blir som å si at «den høyeste i klassen er minst 1 m høy». Det vil si, man gir en *nedre* grense for *maksimum*.

I andre uke er det *to alternative forelesninger*, 2a og 2b. Du kan velge en av dem, med tilhørende øving, eller bruke uken på å jobbe videre med stoffet og øvingen fra første uke.

Det er meningen at du skal kunne velge basert på egne behov og interesser: Øving 1, 2a og 2b har samme frist og kravet til antall godkjente øvinger er satt lavt nok til at man skal kunne velge fritt. Og fagstoffet knyttet til 2a og 2b er *støttelitteratur*, ikke pensum; det vil ikke bli stilt spørsmål direkte om dette stoffet, ut over det som uansett er dekket andre steder.

Du må selvfølgelig gjerne følge begge forelesninger og prøve deg på begge øvinger om du er nysgjerrig og har kapasitet til det!

Se også <https://almdat.idi.ntnu.no/forelesn.html>.

Forelesning 2a

Datastrukturer

For å unngå grunnleggende kjøretidsfeller er det viktig å kunne organisere og strukturere data fornuftig. Her skal vi se på hvordan enkle strukturer kan implementeres i praksis, og hva vi vinner på å bruke dem i algoritmene våre.

Støttelitteratur:

- ☐ Innledningen til del III
- ☐ Kap. 10. Elementary data structures: Innledning, 10.1 og 10.2
- ☐ Kap. 11. Hash tables: Til og med 11.3.1
- ☐ Kap. 16. Amortized analysis: Side 460–463 (tom. «at most 3»)

Læringsmål knyttet til støttelitteratur:

Disse læringsmålene beskriver de sentrale elementene i støttelitteraturen over, og vil ikke bli direkte testet på eksamen, ut over det som dekkes av andre temaer, eller som forventes som grunnleggende forkunnskaper.

- [B₁] Forstå hvordan *tabeller* (*arrays*), *matriser*, *stakker* og *køer* fungerer
- [B₂] Forstå hvordan *lenkede lister* fungerer
- ! [B₃] Forstå hvordan *hashtabeller* fungerer
- [B₄] Vite at man for *statiske datasett* kan ha *worst-case* $O(1)$ for søk (såkalt *perfekt hashing*)
- [B₅] Forstå hvordan *tabelldobling* (*array doubling*) fungerer

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Ikke nødvendigvis veldig vanskelig, men svært viktig å få med seg.

$$1 + 2 + 3 + \cdots + n = n \cdot (n + 1) / 2 \quad \text{trekantttall, aritmetisk rekke}$$

$$1 + 2 + 4 + \cdots + n = 2n - 1 \quad \text{toerpotenser, geometrisk rekke}$$

Se appendiks A for forklaring.

Forelesning 2b

Problemer og reduksjoner

Vi jobber videre med grunnleggende byggesteiner, og skisserer et rammeverk for å tilegne seg resten av stoffet. Vi ser på hvordan enkle algoritmer kan transformere, eller *redusere*, mellom ulike problemer eller delproblemer, og bygge videre på løsninger eller delløsninger.

Støttelitteratur:

□ Appendiks B i dette heftet: B.1

Læringsmål knyttet til støttelitteratur:

Disse læringsmålene beskriver de sentrale elementene i støttelitteraturen over, og vil ikke bli direkte testet på eksamen, ut over det som dekkes av andre temaer.

[B₆] Forstå definisjonene av *søkeproblemer*, *beslutningsproblemer* og *optimeringsproblemer*

[B₇] Forstå ulike typer reduksjoner (*Turing*, *Cook*, *Levin* og *Karp*), og forholdet mellom dem

Forelesningen understøtter også læringsmål [A₇], [C₁] og [M₆].

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Denne uken er utfordringen kanskje mest å tåle litt forvirring, uten å la seg overvelde. Ideene bak reduksjoner, rekursiv dekomponering og kompleksitetsklasser kommer vi tilbake til flere ganger i løpet av semesteret!

For å si det med Rhett Allain (min utheving): «Students are under the impression that when they are stuck and confused, they are doing something wrong. Think of it this way. What if you went to the gym to work out but you didn't get sweaty and you weren't sore or tired? You would probably feel like you really didn't get any exercise. The same is true for learning. **Confusion is the sweat of learning.**»

Forelesning 3

Splitt og hersk

Rekursiv dekomponering er kanskje den viktigste ideen i hele emnet, og designmetoden *splitt og hersk* er en grunnleggende utgave av det: Del instansen i mindre biter, løs problemet rekursivt for disse, og kombiner løsningene.

Pensum:

- ☐ Innledningen til del II
- ☐ Kap. 2. Getting started: 2.3
- ☐ Kap. 4. Divide-and-conquer: Innledning og 4.3–4.5
- ☐ Kap. 7. Quicksort
- ☐ Appendiks C og D i dette heftet

Læringsmål:

- ! [C₁] Forstå *rekursiv dekomponering* og *induksjon over delinstanser*
- ! [C₂] Forstå designmetoden *divide-and-conquer* (*splitt og hersk*)
- [C₃] Forstå BISECT og BISECT' (se appendiks D i dette heftet)
- [C₄] Forstå MERGE-SORT
- [C₅] Forstå QUICKSORT og RANDOMIZED-QUICKSORT
- ! [C₆] Kunne løse rekurrenser med *substitusjon*, *rekursjonstrær* og *masterteoremet*
- ! [C₇] Kunne løse rekurrenser med *iterasjonsmetoden* (se appendiks C i dette heftet)

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: For mange kan nok den rekursive/induktive tankegangen bak rekursiv dekomponering kreve litt tilvenning. Det samme gjelder logikken bak rekurrensregning. Hvis du klarer å se at det egentlig handler om det samme, så har du kommet et godt stykke på vei!

Forelesning 4

Rangering i lineær tid

Vi kan ofte få bedre løsninger ved å styrke kravene til input eller ved å svekke kravene til output. Sortering basert på sammenligninger ($x \leq y$) er et klassisk eksempel: I verste tilfelle må vi bruke $\lg n!$ sammenligninger, men om vi *antar mer* om elementene eller *bare sorterer noen* av dem så kan vi gjøre det bedre.

Pensum:

- ☐ Kap. 8. Sorting in linear time
- ☐ Kap. 9. Medians and order statistics

Læringsmål:

- ! [D₁] Forstå hvorfor *sammenligningsbasert sortering* har en *worst-case* på $\Omega(n \lg n)$
- [D₂] Vite hva en *stabil sorteringsalgoritme* er
- [D₃] Forstå COUNTING-SORT, og hvorfor den er stabil
- ! [D₄] Forstå RADIX-SORT, og hvorfor den trenger en stabil subrutine
- [D₅] Forstå BUCKET-SORT
- [D₆] Forstå RANDOMIZED-SELECT
- [D₇] Kjenne til SELECT

(Inkl. å vite at den kan brukes til å finne de k minste elementene med kjøretid $O(n)$.)

Merk: Det kreves ikke grundig forståelse av virkemåten til SELECT.

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: I læringsmål [D₁] kommer vi tilbake til poenget med at alle asymptotiske operatorer kan brukes på alle kjøretidsfunksjoner/«tilfeller», og her beskrives altså kjøretiden i *verste tilfelle* med en *nedre grense*. Vi kan alltså gjøre det verre, og i *beste tilfelle* kan vi gjøre det bedre.

For at grensen skal «smitte over» på et annet problem, må vi redusere *fra* sortering, med en reduksjon som har kjøretid $o(n \lg n)$. (Skjønner du hvorfor?)

Forelesning 5

Rotfaste trestrukturer

Rotfaste trær gjenspeiler rekursiv dekomponering. I binære søketrær er alt i venstre deltre mindre enn rota, mens alt i høyre deltre er større, og det gjelder rekursivt for alle deltrær! Hauger er enklere: Alt er mindre enn rota. Det begrenser funksjonaliteten, men gjør dem billigere å bygge og balansere.

Pensum:

- ☐ Innledningene til del III og V
- ☐ Kap. 6. Heapsort
- ☐ Kap. 10. Elementary data structures: 10.3
- ☐ Kap. 12. Binary search trees

Læringsmål:

- ! [E₁] Forstå hvordan *hauger* fungerer, inkl. *hauegenskapen* (*the heap property*), og hvordan de kan brukes som *prioritetskøer*
(PARENT, LEFT, RIGHT, MAX-HEAPIFY, BUILD-MAX-HEAP, HEAPSORT, MAX-HEAP-INSERT, MAX-HEAP-EXTRACT-MAX, MAX-HEAP-INCREASE-KEY, MAX-HEAP-MAXIMUM. Også tilsvarende for *min-heaps*, f.eks., BUILD-MIN-HEAP og MIN-HEAP-EXTRACT-MIN.)
- [E₂] Forstå HEAPSORT
- [E₃] Forstå hvordan *rotfaste trær* kan implementeres med noder og pekere (inkl. med pekere til første barn og neste søsken*)
- ! [E₄] Forstå hvordan *binære søketrær* fungerer, inkl. *binær-søketre-egenskapen* (*the binary-search-tree property*)
(INORDER-TREE-WALK, TREE-SEARCH, ITERATIVE-TREE-SEARCH, TREE-MINIMUM, TREE-MAXIMUM, TREE-SUCCESSOR, TREE-PREDECESSOR, TREE-INSERT, TRANSPLANT, TREE-DELETE)
- [E₅] Vite at forventet høyde for et tilfeldig binært søketre er $\Theta(\lg n)$
- [E₆] Vite at det finnes søketrær med garantert høyde på $\Theta(\lg n)$

Merk: Det kreves ikke grundig forståelse av TRANSPLANT og TREE-DELETE.

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Her er det greit å få med seg at BUILD-MAX-HEAP *ikke* er det samme som gjentatt bruk av MAX-HEAP-INSERT, og at det ikke gir samme kjøretid.

* Kun på lysark, ikke i læreboka.

Forelesning 6

Dynamisk programmering

På sett og vis en generalisering av *splitt og hersk*, der delproblemer kan overlappe: I stedet for et *tre* av delinstans-avhengigheter har vi en *rettet asyklisk graf*. Vi finner og lagrer del-løsninger i en rekkefølge som stemmer med avhengighetene.

Pensum:

- ☐ Innledningen til del IV
- ☐ Kap. 14. Dynamic programming: Innledning, 14.1 og 14.3–14.4
- ☐ Appendiks E i dette heftet

Delkapittel 14.2 og 14.5 kan være nyttige som støttelitteratur.

Læringsmål:

- ! [F₁] Forstå ideen om en *delinstansgraf*
- ! [F₂] Forstå designmetoden *dynamisk programmering*
- ! [F₃] Forstå løsning ved *memoisering* (*top-down*)
- [F₄] Forstå løsning ved *iterasjon* (*bottom-up*)
- [F₅] Forstå hvordan man *rekonstruerer* en løsning fra lagrede beslutninger
- [F₆] Forstå hva *optimal delstruktur* er
- [F₇] Forstå hva *overlappende delinstanser* er
- [F₈] Forstå eksemplene *stavkapping* og *LCS*
- [F₉] Forstå løsningen på *det binære ryggsekkproblemet* (appendiks E i dette heftet)
(KNAPSACK, KNAPSACK')

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Å forstå dynamisk programmering handler i stor grad om å forstå hva overlappende delinstanser er. Om du bare har en litt ullen forståelse av det, er det lurt å jobbe mer med å forstå det ordentlig! Det betyr altså at vi har dekomponert i flere delinstanser som (direkte eller indirekte) bygger på samme delinstans. Om dette skjer i flere nivåer, vil en rekursiv løsning fort få eksponentiell kjøretid.

Forelesning 7

Grådighet

Grådige algoritmer består av en serie med valg, og hvert valg tas *lokalt*: Algoritmen gjør alltid det som ser best ut her og nå, uten noe større perspektiv. Slike algoritmer er ofte enkle; utfordringen ligger i å finne ut om de gir rett svar.

Pensum:

- Kap. 15. Greedy algorithms: Innledning og 15.1–15.3

Læringsmål:

- ! [G₁] Forstå designmetoden *grådighet*
- ! [G₂] Forstå *grådighetsegenskapen* (*the greedy-choice property*)
- [G₃] Forstå eksemplene *aktivitetsutvalgelse* og *det kontinuerlige ryggsekkproblemet*
- [G₄] Forstå HUFFMAN og *Huffman-koder*

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Det er lett å tenke at grådighetsegenskapen bare er at vi trygt kan ta grådige valg, og ende opp med en optimal løsning, men det er konklusjonen vi får dersom vi *også* har optimal delstruktur! Grådighetsegenskapen sier bare at vi kan gjøre *ett* grådig valg, og fortsatt ha mulighet til å få en optimal løsning, om vi gjør ting riktig (på et eller annet vis) videre.

Forelesning 8

Traversering av grafer

Vi traverserer en graf ved å besøke noder vi vet om. Vi vet i utgangspunktet bare om start-noden, men oppdager naboene til dem vi besøker. Traversering er viktig i seg selv, men danner også ryggraden til flere mer avanserte algoritmer.

Pensum:

- ☐ Innledningen til del VI
- ☐ Kap. 20. Elementary graph algorithms
- ☐ Appendiks F i dette heftet

Læringsmål:

- [H₁] Forstå hvordan grafer kan implementeres
- [H₂] Forstå BFS, også for å finne *korteste vei uten vektor*
(Inkl. PRINT-PATH)
- [H₃] Forstå DFS, *parentesteoremet* og *hvit-sti-teoremet*
- [H₄] Forstå hvordan DFS *klassifiserer kanter*
- [H₅] Forstå TOPOLOGICAL-SORT
- [H₆] Forstå STRONGLY-CONNECTED-COMPONENTS
- [H₇] Forstå hvordan DFS kan *implementeres med en stakk*
- [H₈] Forstå hva *traverseringstrær* (som *bredde-først-* og *dybde-først-trær*) er
- ! [H₉] Forstå *traversering med vilkårlig prioritetskø*

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Ikke nødvendigvis en *snubletråd*, men ... her kan det være lurt å prøve å forstå forholdet mellom topologisk sortering vha. DFS, og rekursiv løsning av delinstanser i en memoisert algoritme. Delinstansene må jo løses i topologisk sortert rekkefølge, og vi oppnår det på nøyaktig samme måte!

Forelesning 9

Minimale spenntreer

Her har vi en graf med vektorer på kantene, og ønsker å bare beholde akkurat de kantene vi må for å koble sammen alle nodene, med en så lav vektsum som mulig. Erke-eksempel på grådighet: Velg én og én kant, alltid den billigste lovlige.

Pensum:

- ☐ Kap. 19. Data structures for disjoint sets: Innledning, 19.1 og 19.3
- ☐ Kap. 21. Minimum spanning trees

Læringsmål:

- [I₁] Forstå *skog*-implementasjonen av *disjunkte mengder*
(CONNECTED-COMPONENTS, SAME-COMPONENT, MAKE-SET, UNION, LINK, FIND-SET)
- [I₂] Vite hva *spenntreer* og *minimale spenntreer* er
- ! [I₃] Forstå GENERIC-MST
- [I₄] Forstå når og hvorfor *lette kanter* er *trygge kanter*
- [I₅] Forstå MST-KRUSKAL
- [I₆] Forstå MST-PRIM, inkl. kjøretid med Fibonacci-haug

Merk: Det forventes ikke at dere skal vite hvordan Fibonacci-hauger fungerer.

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Hvis du forstår hvorfor det er trygt å velge letteste kant over et snitt som respekterer (dvs., ikke krysser noen av kantene i) løsningen du har bygget så langt, så har du et godt grunnlag for å forstå mye av stoffet denne uken.

Forelesning 10

Korteste vei fra én til alle

Bredde-først-søk kan finne stier med færrest mulig kanter, men hva om kantene har ulik lengde? Det generelle problemet er uløst, men vi kan løse problemet med gradvis bedre kjøretid for grafer (1) uten negative sykler; (2) uten negative kanter; og (3) uten sykler. Og vi bruker samme prinsipp i alle tre tilfeller!

Pensum:

- Kap. 22. Single-source shortest paths: Innledning og 22.1–22.3

Læringsmål:

- [J₁] Forstå ulike varianter av *korteste-vei-* eller *korteste-sti-*problemet
(*Single-source, single-destination, single-pair, all-pairs*)
- [J₂] Forstå strukturen til *korteste-vei-*problemet
- [J₃] Forstå at *korteste enkle vei* (*simple path*) kan eksistere selv om en negativ sykel forhindrer at *korteste vei* gjør det
- [J₄] Forstå reduksjoner mellom *korteste enkle vei* og *lengste enkle vei*
- [J₅] Forstå hvordan man kan representere et *korteste-vei-tre*
- ! [J₆] Forstå *kant-oppdatering* (*edge relaxation*) og RELAX
- [J₇] Forstå ulike egenskaper ved korteste veier og oppdatering
(*Triangle inequality, upper-bound property, no-path property, convergence property, path-relaxation property, predecessor-subgraph property*)
- [J₈] Forstå BELLMAN-FORD
- [J₉] Forstå DAG-SHORTEST-PATHS
- ! [J₁₀] Forstå kobling mellom DAG-SHORTEST-PATHS og dynamisk programmering
- [J₁₁] Forstå DIJKSTRA, inkl. kjøretid med Fibonacci-haug

Merk: Det forventes ikke at dere skal vite hvordan Fibonacci-hauger fungerer.

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Anta at det finnes en sti fra s til t . Den korteste stien, om den finnes, er alltid enkel, og kan finnes i polynomisk tid. Den korteste enkle stien finnes alltid, men vi kan ikke finne den i polynomisk tid (med mindre $P = NP$).

Forelesning 11

Korteste vei fra alle til alle

Vi kan finne korteste veier fra hver node etter tur, men mange delinstanser vil overlappe; om vi har mange kanter, lønner det seg å bruke dynamisk programmering. Vi dekomponerer basert på hvor mange kanter vi får bruke i løsningen, eller hvilke noder som får inngå.

Pensum:

□ Kap. 23. All-pairs shortest paths: Innledning og 23.2

Læringsmål:

- [K₁] Forstå *forgjengerstrukturen* for *alle-til-alle*-varianten av korteste-vei-problemet
(PRINT-ALL-PAIRS-SHORTEST-PATH)
- [K₂] Forstå FLOYD-WARSHALL
- [K₃] Forstå TRANSITIVE-CLOSURE

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \implies \pi_{ij}^{(k)} = \pi_{kj}^{(k-1)}$

Forelesning 12

Maksimal flyt

Et stort skritt i retning av generell lineær optimering (såkalt lineær programmering). Her ser vi på to tilsynelatende forskjellige problemer, som viser seg å være *duale* av hverandre, noe som hjelper oss med å finne en løsning.

Pensum:

- Kap. 24. Maximum flow

Læringsmål:

- [L₁] Kunne definere *flytnett*, *flyt* og *maks-flyt-problemet*
- [L₂] Kunne håndtere *antiparallelle kanter* og *flere kilder og sluk*
- ! [L₃] Kunne definere *restnettet* til et flytnett med en gitt flyt
- [L₄] Forstå hvordan man kan *oppheve* (*cancel*) flyt
- [L₅] Forstå hva en *forøkende sti* (*augmenting path*) er
- [L₆] Forstå hva *snitt*, *snitt-kapasitet* og *minimalt snitt* er
- ! [L₇] Forstå *maks-flyt/min-snitt-teoremet*
- [L₈] Forstå FORD-FULKERSON-METHOD og FORD-FULKERSON
- [L₉] Forstå *Edmonds-Karp*-algoritmen (FORD-FULKERSON med BFS)
- [L₁₀] Forstå hvordan FORD-FULKERSON kan finne et minimalt snitt
- [L₁₁] Forstå hvordan maks-flyt kan finne en *maksimum bipartitt matching*
- ! [L₁₂] Forstå *heltallsteoremet* (*integrality theorem*)
- ! [L₁₃] Være i stand til å konstruere reduksjoner til maks-flyt-problemet

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Ligning 24.2.

Forelesning 13

NP-komplettethet

NP er den enorme klassen av *ja-nei*-problemer der ethvert ja-svar har et bevis som kan sjekkes i polynomisk tid. Alle problemer i NP kan i polynomisk tid reduseres til de såkalt *komplette* problemene i NP. Dermed kan ikke disse løses i polynomisk tid, med mindre *alt* i NP kan det. Ingen har klart det så langt...

Pensum:

- ☐ Kap. 34. NP-completeness: Innledning og 34.1–34.3
- ☐ Appendiks E i dette heftet

Støttelitteratur:

- ☐ Appendiks B i dette heftet: B.2

Læringsmål:

- [M₁] Forstå *koding* (*encoding*) av en instans
- [M₂] Forstå hvorfor løsningen på det binære ryggsekkproblemet *ikke er polynomisk*
- [M₃] Forstå forskjellen på *konkrete* og *abstrakte* problemer
- [M₄] Forstå representasjonen av beslutningsproblemer som *formelle språk*
- [M₅] Forstå definisjonen av klassene P, NP og co-NP
- [M₆] Forstå *redusibilitets-relasjonen* som relativ vanskegrad
- ! [M₇] Forstå definisjonen av NP-hardhet og NP-komplettethet
- [M₈] Forstå forholdet mellom NP-hardhet/NP-komplettethet og ulike problemtyper
(Søkeproblemer, beslutningsproblemer og optimeringsproblemer)
- [M₉] Forstå den konvensjonelle hypotesen om forholdet mellom P, NP og NPC
- [M₁₀] Forstå reduksjon begge veier mellom optimering og terskling
- [M₁₁] Forstå reduksjon begge veier mellom søk og beslutning
- [M₁₂] Forstå beviset for at CIRCUIT-SAT er NP-komplett

Merk: Det kreves ikke grundig forståelse av NP-komplettetsbeviset.

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd: Om vi vil vise at X er vanskelig, må vi redusere *til* X!

Forelesning 14

NP-komplette problemer

Om du står overfor et NP-komplett problem, så er det viktig å kunne bevise det. Vi har sett at alt i NP kan reduseres til CIRCUIT-SAT direkte, men vi kan også vise kompletthet indirekte, ved å redusere videre fra CIRCUIT-SAT!

Pensum:

- Kap. 34. NP-completeness: 34.4 og 34.5

Læringsmål:

- ! [N₁] Forstå hvordan NP-kompletthet kan bevises ved én reduksjon
- ! [N₂] Kjenne de NP-komplette problemene CIRCUIT-SAT, SAT, 3-CNF-SAT, CLIQUE, VERTEX-COVER, HAM-CYCLE, TSP og SUBSET-SUM
- [N₃] Forstå NP-kompletthetsbevisene for disse problemene
- [N₄] Forstå at det binære ryggsekkproblemet er NP-hardt
- [N₅] Forstå at *lengste-enkle-vei*-problemet er NP-hardt
- [N₆] Kjenne generelle reduksjonsstrategier og kunne konstruere enkle bevis for NP-hardhet og NP-kompletthet

Merk: Det kreves ikke grundig forståelse av de ulike NP-kompletthetsbevisene.

Husk også læringsmål Z₁–Z₁₀ på side v, som gjelder hver forelesning.

Ukens snubletråd (reprise): Om vi vil vise at X er vanskelig, må vi redusere *til* X!

Appendiks A

Et par gjengangere

Et par summer dukker opp flere ganger i emnet. Disse er det lurt å få i fingrene tidlig!

Håndtrykksformelen

Dette er en av to enkle, men viktige formler som stadig dukker opp når man skal analysere algoritmer. Formelen er:

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

Den kalles gjerne *håndtrykksformelen*, fordi den beskriver antall håndtrykk som utføres om n personer skal hilse på hverandre.* Vi kan vise dette ved å telle antall håndtrykk på to ulike måter. De to resultatene må da være like.

Telling 1: La oss se på personene én etter én. Første person hilser på de andre $n-1$ personene. Andre person har allerede hilst på den første, men bidrar med $n-2$ nye håndtrykk ved å hilse på de gjenværende. Generelt vil person i bidra med $n-i$ nye håndtrykk, så totalen blir

$$(n-1) + (n-2) + \dots + 2 + 1 + 0.$$

Om vi snur summen, så vi får $0 + 1 + \dots + (n-1)$, så er dette den venstre siden av ligningen. Dette oppstår typisk i algoritmer de vi utfører en løkke gjentatte ganger, og antall iterasjoner i løkken øker eller synker med 1 for hver gang, slik:

```
1 for i = 1 to n - 1
2     for j = i + 1 to n
3         i tar j i hånden
```

Telling 2: Vi kan også telle på en mer rett frem måte: Hver person tar alle de andre i hånden, og inngår dermed i $n-1$ håndtrykk. Hvis vi bare teller hver enkelt person sin halvdel av håndtrykket, får vi altså $n(n-1)$ halve håndtrykk. To slike halve håndtrykk utgjør jo ett helt, så det totale antallet håndtrykk blir den høyre siden av ligningen, nemlig $n(n-1)/2$.

Man kan også gjøre om en sum av denne typen ved å brette den på midten, og legge sammen første og siste element, nest første og nest siste, etc. Vi får da

$$(n-1+0) + (n-2+1) + (n-3+2) + \dots$$

Hvert ledd summerer til $n-1$, og det er $n/2$ ledd. Mer generelt er summen av en aritmetisk rekke (der vi øker med en konstant fra ledd til ledd) lik gjennomsnittet av første og siste ledd, multiplisert med antallet ledd i rekken.

Om vi summerer fra 1 til n , heller enn fra 0 til $n-1$, blir svaret $n(n+1)/2$.

* Summene kalles også *trekantall*.

Utslagsturneringer

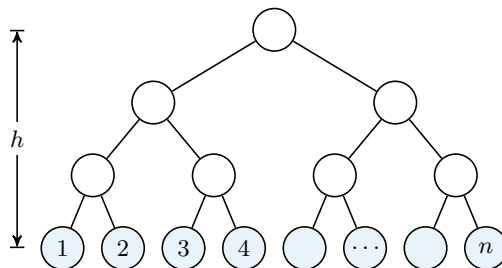
Dette er den *andre* av de to sentrale formlene:

$$\sum_{i=0}^{h-1} 2^i = 2^h - 1$$

Det vil si, de første toerpotensene summerer til én mindre enn den neste. En måte å se dette på er av totalssystemet, der et tall $a = 11 \dots 1$ med h ettall etterfølges av tallet $b = 100 \dots 0$, som består av ett ettall, og h nuller. Her er $a = 2^0 + 2^1 + \dots + 2^{h-1}$ og $b = 2^h$, og $a = b - 1$.

Et grundigere bevis kan vi få ved å bruke samme teknikk som før, og telle samme ting på to ulike måter. Det vi vil telle er antall matcher i en utslagsturnering (*knockout*-turnering), det vil si, en turnering der taperen i en match er ute av spillet. Dette blir altså annerledes enn såkalte *round robin*-turneringer, der alle møter alle – for dem kan vi bruke håndtrykksformelen til å finne antall matcher, siden hver match tilsvarer ett håndtrykk.

Telling 1: Vi begynner med å sette opp et *turneringstre*:



Her er deltakerne plassert nederst, i løvnodene; hver av de tomme, hvite nodene representerer en match, og vil etterhvert fylles med vinneren av de to i nodene under, helt til vi står igjen med én vinner på toppen. Vi går altså gjennom h runder, og i hver runde organiseres de spillerne som gjenstår i par som skal møtes. Om vi nummererer rundene baklengs, etter hvor mange runder det er igjen til finalen, så vil antall matcher i runde i være 2^i . Det totale antall matcher er altså $2^0 + 2^1 + \dots + 2^{h-1}$, som er venstresiden av ligningen vår.

Telling 2: Antall deltakere er $n = 2^h$. I hver match blir én deltaker slått ut, helt til vi sitter igjen med én vinner etter finalen. Antall matcher vi trenger for å slå ut alle bortsett fra én er $n - 1$, eller $2^h - 1$, som er høyresiden i ligningen.

Det er forøvrig også viktig å merke seg *høyden til treet*: $h = \log_2 n$. Dette er altså antall doblinger fra 1 til n , eller antall halvinger fra n til 1. Det er også en sammenheng som vil dukke opp ofte.

En mer generell summeformel finner vi i appendix A.1 i læreboka (ligning A.6):

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1} \quad (x \neq 1)$$

Denne er ganske enkel å huske, om du husker tilfellet for $x = 2$. Det eneste ekstra elementet er divisoren, og for at den skal bli 1 for $x = 2$, «må» den nesten være $x - 1$.

Det er også verdt å merke seg at det siste leddet i rekken dominerer når $x > 1$:

$$\sum_{k=0}^n x^k = \Theta(x^n) \quad (x > 1)$$

Appendiks B

Problemer og reduksjoner

Dette kapittelet dukker opp tidlig i semesteret, men kommer så vidt inn på noen ganske avanserte temaer, som diskuteres grundigere i de siste forelesningene. Fokuser i første omgang på delkapittel B.1. Prøv å få med deg hovedpoengene, og plukk det gjerne opp igjen senere!

B.1 Relasjoner, problemer og reduksjoner

Problemer og reduksjoner diskuteres og defineres i læreboka, men det skjuler seg noen varianter under overflaten der. Å se litt nærmere på disse variantene kan gjøre flere ting lettere å forstå. Dette appendikset går ikke veldig i dybden, og er mest ment som en opprydding og begrepsavklaring. Om man er nysgjerrig på flere detaljer, kan f.eks. boka *P, NP, and NP-Completeness* av Goldreich [3] være en grei innføring.

Relasjoner og problemer

Når vi snakker om et *problem*, mener vi et *generelt* problem, som f.eks. «sortering». Vi kan så se på spesifikke tilfeller, eller *instanser*, av problemet, som «sorter tallene 6, 3 og 9». En *instans* er én bestemt *input*, og til hver instans hører et sett med riktige løsninger, eller *outputs*.

Når vi er litt mindre formelle, og det ikke er rom for misforståelse, hender det vi også bruker begrepet *problem* om en bestemt instans.

Hva som er riktig svar for hver instans kan vi uttrykke som en *binær relasjon* R mellom input og output, altså et sett med input-output-par. I formelle sammenhenger, tenker vi gjerne på input og output som bitstrenger, så vi har $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$. Her står $\{0, 1\}^*$ for mengden av mulige sekvenser med elementer fra mengden $\{0, 1\}$.

I praksis snakker vi gjerne om relasjoner mellom mer vilkårlige mengder, som f.eks. mengden av heltallssekvenser eller rettede grafer. Det er *tolkningene* våre av disse bitstrengene.

I boka snakker de om de bitstreng-kodede problemene som *konkrete*, mens de mer abstrakte tolkningene omtales som *abstrakte problemer*.

For $x, y \in \{0, 1\}^*$ har vi altså $(x, y) \in R$ dersom y er et riktig svar for x . Vi kan ha flere riktige svar, eller i noen tilfeller *ingen*. Om vi står overfor en binær relasjon R mellom instanser og riktige svar, og har oppgitt en input x , er det særlig tre ting vi ønsker å gjøre:*

1. Finne en output y slik at $(x, y) \in R$, hvis det eksisterer.
2. For en gitt output y , verifisere at $(x, y) \in R$.
3. Avgjøre om det eksisterer en output y slik at $(x, y) \in R$.

* Dette kalles gjerne ulike *computational tasks*, eller *beregningsoppgaver*. Det finnes andre slike beregningsoppgaver, som f.eks. å telle antall løsninger, men det kommer vi ikke inn på her.

Det å finne en løsning er kanskje det mest naturlige. Det kaller vi *søkeproblemet* (*the search problem*) knyttet til relasjonen R. Når vi tolker eller omtaler en relasjon som et problem i seg selv, er det dette vi mener. En algoritme som *løser* et slikt problem må altså produsere en riktig output for hver input der det er mulig, dvs. hver instans der det faktisk finnes et svar. Dersom en gitt instans ikke har noen løsning, krever vi at algoritmen gir beskjed om det, f.eks. gjennom en feilmelding eller en spesiell returverdi.

Det kan være upraktisk å håndtere alle mulige bitstrenger som input. Ofte bryr vi oss bare om noen av dem, og definerer alle andre som *ugyldige*. Korrekthet og kjøretid gjelder da bare gyldige instanser. For ugyldig input kan en algoritme gjøre hva den vil; den trenger ikke engang terminere!

Hvis det er lett å sjekke om input er gyldig, kan vi også omdefinere ugyldig input til *gyldige instanser uten løsning*, og utvide algoritmene våre til å håndtere all input.

Å *finne* et riktig svar for en instans er ikke det eneste vi kan gjøre. Vi kan også ønske å *verifisere* at et svar er riktig. La oss si vi har fått oppgitt en instans $\langle 6, 3, 9 \rangle$ som skal sorteres, og en påstått løsning $\langle 3, 6, 9 \rangle$. Vår oppgave er da å sjekke om denne påståtte løsningen er en sortert utgave av instansen. En algoritme som gjør dette, kaller vi en *verifikasjonsalgoritme* for relasjonen (og problemet). En enkel prosedyre for å verifisere sortering er, for eksempel:

```
IS-SORTED(A, n)
1  for i = 1 to n - 1
2      if A[i] > A[i + 1]
3          return FALSE
4  return TRUE
```

Dette verifiserer bare at A er sortert. En fullverdig verifikasjonsalgoritme sjekker *forholdet mellom input og output* (altså instans og løsning). I dette tilfellet, kunne vi ha sett på A før og etter sortering, men det enkleste er om vi tenker på løsningen som en *permutasjon* av A, altså en tabell $\pi[1:n]$ med verdier $1, \dots, n$ som angir hvilket element som skal flyttes til hver posisjon i A. Vi kan da verifisere at en slik permutasjon faktisk sorterer A:

```
SORTS(A,  $\pi$ , n)
1  for i = 1 to n - 1
2      if A[ $\pi[i]$ ] > A[ $\pi[i + 1]$ ]
3          return FALSE
4  return TRUE
```

Strengt tatt burde vi også verifisere at π faktisk er en permutasjon, altså at hver verdi $1, \dots, n$ finnes i π nøyaktig én gang. Dette kan gjøres i lineær tid med en boolsk tabell.

Kjøretiden til SORTS er $\Theta(n)$. Vi skal se (i forelesning 4) at å faktisk *finne* en slik permutasjon krever kjøretid $\Omega(n \lg n)$ i verste tilfelle, om vi ikke gjør spesielle antagelser om input. Med andre ord ser vi at det er *lettere å verifisere løsningen enn å finne den*.*

Når vi beveger oss inn på området *kompleksitetsteori* (og NP-komplekthet og lignende), dukker det opp en annen problemtype som kan være litt ryddigere å forholde seg til, selv om den er mindre anvendelig i praksis. Der består problemet av et spørsmål, og vi vil bare

* Det åpne spørsmålet om $P \neq NP$, som vi kommer til, handler om samme type problemstilling, men der går skillet mellom polynomisk og ikke-polynomisk kjøretid. Altså, finnes det problemer der vi kan trenge mer enn polynomisk tid på å finne løsningen, selv om den kan verifiseres i polynomisk tid?

ha svaret «ja» eller «nei». Vi *kan* tenke på dette som et søkeproblem med bare to mulige løsninger for hver instans, men det er vanlig å forenkle dette, og definere problemet som en *delmengde av instansene*, altså de instansene der «svaret er ja». Oppgaven vår er å avgjøre om en gitt instans tilhører denne delmengden. Slike problemer kaller vi **beslutningsproblemer**.

Om vi ser på instansene som strenger, så vil beslutningsproblemer tilsvare det som formelt kalles et *språk*.

En spesielt viktig type beslutningsproblemer følger direkte fra relasjonene vi har brukt til å definere søkeproblemer, ved at vi stiller spørsmålet: «Har denne instansen en løsning?» Med andre ord:

- Beslutningsproblemet er å avgjøre om noe finnes.
F.eks.: *Har denne grafen en Hamilton-sykel?*
- Søkeproblemet er å faktisk finne det.
F.eks.: *Finn en Hamilton-sykel i denne grafen.*

Vi sier gjerne at beslutningsproblemet er *implisitt i* relasjonen eller søkeproblemet.

Vi kan også *starte* med et beslutningsproblem, og så *finne opp* en relasjon. Da kaller vi gjerne relasjonen en **vitnerelasjon**, og bruker ord som *vitne*, *bevis* eller **sertifikat** heller enn *løsning* eller *svar* om høyresiden, siden vi kan bruke den til å bevise at svaret skal være ja. Oppførselen er den samme: Instansene vi er ute etter (der svaret er «ja») er akkurat dem der det finnes et slikt sertifikat; altså instansene som «har en løsning». Søkeproblemet vårt blir altså: «Finnes det et bevis for at dette er en ja-instans?» For eksempel vil det å finne en Hamilton-sykel i en graf opplagt bevise at grafen har en Hamilton-sykel.

Hvis vi har et beslutningsproblem, kan vi definere en tilhørende relasjon der eneste riktige løsning er f.eks. «OK». Om svaret på beslutningsproblemet skal være «ja», kan vi si at løsningen er «OK», og om svaret skal være «nei», finnes ingen løsning.

Dette fungerer generelt, men vi vil gjerne ha *bedre* eller mer naturlige relasjoner enn dette, der vi faktisk kan finne en verifikasjonsalgoritme! Det vil si, vi vil ha et mer ordentlig, verifiserbart bevis for at svaret er «ja».

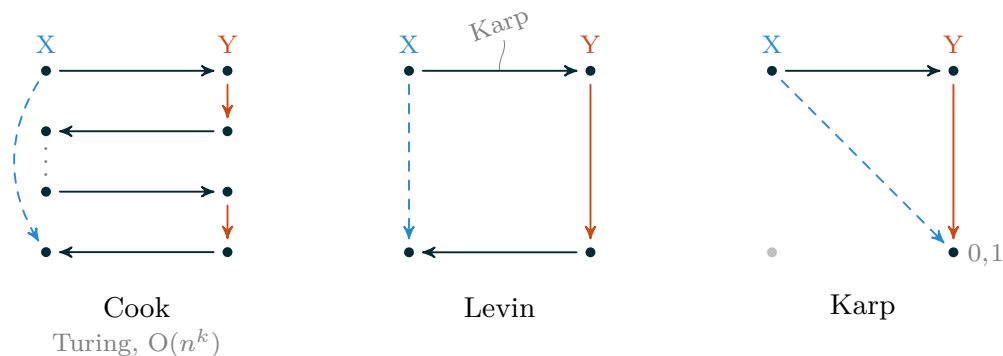
Dette er grunnlaget for klassen NP, som vi kommer tilbake til litt senere.

Generelle reduksjoner: Turing og Cook

Det er ikke alltid vi kan løse et problem direkte, men vi kan kanskje gjøre noe arbeid, så vi sitter igjen med en *redusert utgave* av problemet. Dette arbeidet kaller vi en **reduksjon**. La oss si at vi vil vite om alle n elementer i en tabell A er unike (dvs., forskjellige). Vi har begynt på en implementasjon:

```
UNIQUE( $A, n$ )
1  INSERTION-SORT( $A, n$ )
2  // TODO: Er naboverdier forskjellige?
```

Ved å sortere, har vi *redusert* problemet til å avgjøre om naboverdier $A[i]$ og $A[i + 1]$ er forskjellige, for $i = 1, \dots, n - 1$. Å gjøre denne sjekken er arbeidet som gjenstår, markert med en **TODO**-kommentar, som er en vanlig konvensjon når koden ikke er helt ferdig.



Figur B.1: Typer reduksjoner med polynomisk kjøretid, fra X til Y. De svarte pilene er arbeidet som utføres av reduksjonen. De røde pilene representerer orakelet, altså en hypotetisk løsning på problemet Y. De blå, stiplede pilene er løsningene vi får på X, om vi kombinerer de andre pilene.

Arbeidet vi faktisk har implementert, altså å sortere A, er reduksjonen, og den fullstendige prosedyren UNIQUE, som inkluderer en hypotetisk implementasjon av linje 2, gir oss en løsning på det opprinnelige problemet vårt. Hvis vi kaller det opprinnelige problemet X, har vi altså *reduisert fra X til Y*, der Y er å avgjøre om naboverdier er forskjellige.

En måte å se dette på er at prosedyren vår er en underspesifisert (eller «magisk») løsning på problemet, som benytter seg av et «orakel» for den biten som mangler. Et slikt *orakel* er en hypotetisk (muligens «magisk») løsning på det problemet vi har redusert til. *Kjøretiden* til reduksjonen får vi om vi antar at orakelet har kjøretid $O(1)$, og reduksjonen er *riktig* hvis vi får rett svar så lenge orakelet svarer rett.

I utgangspunktet kan vi bruke orakelet flere ganger, som tilsvarer at vi har flere *TODO*-kommentarer (eller kanskje én kommentar inne i en løkke). Vi kan også redusere til flere problemer, ved å bruke flere orakler.* Dette er den mest generelle formen for reduksjoner vi ser på. Hvis vi skal være ekstra tydelige, kaller vi dem gjerne *Turing-reduksjoner*, etter Alan Turing. Turing var primært interessert i om ting kunne løses i det hele tatt, mens vi er spesielt interessert i *effektive* reduksjoner, siden disse potensielt kan gi oss effektive løsninger. Vi fokuserer primært på reduksjoner med *polynomisk kjøretid*, som vi kaller *Cook-reduksjoner*, etter Stephen Cook. Du finner et diagram som illustrerer en slik reduksjon i venstre panel i figur B.1.

Figur B.1 viser egentlig Cook- og Levin-reduksjoner som er *litt snevrere* enn det definisjonene tilsier. Arbeidet som utføres av pilene lenger ned kan nemlig også basere seg på det som skjer lenger opp! For eksempel, har «tilbake-pilen» i en Levin-reduksjon *også tilgang til opprinnelig input*. All koden i reduksjonene har altså tilgang til det som har skjedd tidligere, uavhengig av om vi bruker et orakel innimellom.

Reduksjoner som transformasjoner: Levin og Karp

Et vanlig tilfelle er at vi *kun tillater én TODO-kommentar*, eller altså én forespørsel til orakelet vårt. Vi kan da se på reduksjonen som et par med *transformasjoner*: *Fremover* for input og *bakover* for output. Vi transformerer altså den opprinnelige instansen av problem X til en instans av et annet problem Y, og så beskriver vi hvordan vi transformerer en hypotetisk Y-løsning tilbake til en X-løsning.† En slik reduksjon med polynomisk kjøretid kaller vi en *Levin-reduksjon*, etter Leonid Levin.

* Dette kan vi formelt sett se på som ett orakel for et kombinert problem, om vi foretrekker det.

† Denne transformasjonen tilbake har også tilgang til opprinnelig input.

Merk at vi her *krever* at enhver Y-løsning *skal* produsere en X-løsning. Vi har ikke lov til å kaste bort Y-løsningen og deretter si at vi ikke har noen løsning på X! Hvis vi gjør det (som f.eks. i FIND-SHORT-PATH på side 27) kaller vi det *ikke* en Levin-reduksjon.

Om vi begrenser oss til beslutningsproblemer, er transformasjonen tilbake mindre interessant.* Vi returnerer da vanligvis bare svaret fra problem Y direkte, som i prosedyren UNIQUE, beskrevet tidligere. En slik reduksjon, der *TODO-kommentaren vår kommer helt til slutt*, kaller vi en **Karp-reduksjon**, etter Richard M. Karp.[†]

Karp-reduksjoner er den typen reduksjoner som eksplisitt diskuteres i læreboka (se f.eks. figur 34.1), og **er dem vi bruker til å definere NP-komplekthet**.

Noe å tenke på: Om vi har en Levin-reduksjon for et søkeproblem, så vil funksjonen som transformerer instansene *være en Karp-reduksjon for det tilhørende beslutningsproblemet*, som angitt i midterste panel i figur B.1. Ser du hvorfor?

Reduksjoner og vanskegrad

En ting som blir sentralt særlig mot slutten av semesteret, er å bruke reduksjoner, ikke til å *løse* problemer, men til å *vise at de er vanskelige*. Det er altså det *motsatte* av det vi normalt ville ha gjort, og da gjelder det å holde tunga rett i munnen: For at det faktisk skal bli omvendt, så må vi *redusere motsatt vei*!

Poenget er at vi vil sammenligne problemet vårt X med et annet, vanskelig problem Y, og da *nytter det ikke* å redusere *til* Y, som dette:

SOLVE-X(*args*)

- 1 do some work to reduce to Y
- 2 // TODO: Solve problem Y

En slik reduksjon kan være nyttig hvis vi faktisk har en løsning på Y, men hva om vi *ikke* har det? Det eneste det forteller oss er at vi ikke kan få ferdig prosedyren SOLVE-X *på denne måten*. Men det betyr jo ikke at det ikke finnes *andre* måter å løse X på!

Selv om vi kan redusere til et problem vi ikke har løsning på, så kan det være vi også kan redusere til *andre* problemer, som vi *har* løsninger på.

La oss ta et litt mer spesifikt eksempel:

DO-ALGDAT-EXERCISE

- 1 smash laptop with hammer
- 2 // TODO: Do almdat exercise with smashed laptop

Dette fungerer antagelig dårlig. Vi har startet med ett problem (gjør en almdat-øving) og redusert det til et annet (gjør en almdat-øving med knust laptop), som vi ikke har noen løsning

* Det finnes bare to mulige transformasjoner (identitet eller negasjon), og begge gir oss samme informasjon.

[†] Levin- og Karp-reduksjoner defineres gjerne til å bestå av funksjonene som transformerer input (begge) og output (Levin). Det er jo litt annerledes enn Cook-reduksjoner, som heller defineres som en prosedyre (eller Turing-maskin) med tilgang til et orakel, men de gir begge trivielt opphav til Cook-reduksjoner. For å si det med Goldreich: «Being slightly inaccurate but essentially correct, we shall say that [Levin- and] Karp-reductions are special cases of Cook-reductions.»

på. Men gir det oss noen informasjon? Nei, for det kan jo tenkes, heeelt hypotetisk, at det finnes andre måter å gjøre en algdato-øving på. Det er ikke sikkert vi *må* knuse datamaskinen vår først.

Nei, vi må snu ting på hodet, og *reducere motsatt vei*. Dette kan virke litt bakvendt til å begynne med, så det gjelder å bruke litt mentale krefter på å forstå hvorfor, og ikke minst huske retningen:

For å vise at et problem er vanskelig, kan vi redusere **fra et vanskelig problem**.

Resonnementet følger en slags «for godt til å være sant»-logikk. Det vil si, vi lurte på om vi kan løse problemet vårt, og vil komme frem til at det ville være for godt til å være sant.

Tenk deg at du får et tilbud om å lære et merkelig triks for å bli styrtrik over natta. Alt du trenger gjøre er å betale hundre kroner for informasjonen:

GET-RICH-QUICK

- 1 pay 100 kr
- 2 // TODO: This one weird trick

Her vil vel de fleste være litt skeptiske til om dette merkelige trikset kan ha noe for seg ... i hvert fall at det kan være *enkelt*. Dersom selve reduksjonen (altså resten, utenom den manglende biten) ikke var enkel, kunne vi kanskje også ha trodd på det. For eksempel:

GET-RICH-QUICK

- 1 win the lottery
- 2 // TODO: This one not quite so weird trick

Hvis vi først har vunnet i Lotto, så kan vi jo tro på at det som gjenstår ikke trenger være så vanskelig, f.eks. bare å hente ut pengepremien.

For å ta et litt mindre hypotetisk eksempel, la oss se på følgende to problemer.

PARTITION, eller *fordelingsproblemet*: Sjekk om et sett med heltall kan fordeles i to mengder med samme sum.

TSP, eller *handelsreiseproblemet*: Finn en kortest mulig reiserute som besøker et sett med byer nøyaktig én gang hver, og ender der du startet.

Det første virker kanskje ikke så vanskelig, mens det andre er et problem som er kjent for å være uløst; det er ingen som har funnet noen effektiv algoritme for det, og mange tror det er umulig. Om vi reduserer fra PARTITION til TSP, blir det litt som å prøve å løse en øving ved å knuse laptopen med en hammer; ikke spesielt informativt. Men hva om du får se en reduksjon av denne typen?

SOLVE-TSP

- 1 do a small amount of work
- 2 // TODO: Solve PARTITION
- 3 do a little more work

Omtrent en slik reduksjon finnes faktisk.* Hva tenker du da om PARTITION? Virker det fortsatt som om det er et enkelt problem? Det ville jo i så fall, indirekte, bety at også TSP er et enkelt problem, siden vi kan fullføre prosedyren SOLVE-TSP ved å løse PARTITION! Og ... det ville jo være for godt til å være sant.

Altså: Om vi kan redusere fra X til Y, så kan ikke X være vanskeligere enn Y, og vi skriver $X \leq Y$, som da rett og slett betyr «X kan reduseres til Y». Vi kan også legge på noen ekstra symboler for å indikere typen reduksjon. For eksempel kan vi skrive $X \leq_P Y$ for reduserbarhet i polynomisk tid.†

* Strengt tatt må vi nok bruke orakelet flere ganger – men bare et polynomisk antall.

† Symbolet $\leq_P$ brukes ofte spesifikt om Karp-reduksjoner.

B.2 NP-komplettethet

Reduksjoner gir oss i seg selv bare en *relativ* vanskegrad, men vi kan bruke ideen til å sammenligne med mange problemer på én gang! Om vi har en mengde eller klasse problemer S som alle kan reduseres til et problem X (med en bestemt type reduksjoner), så kan vi si at problemet X er *S-vanskelig* eller *S-hardt*, siden det er minst like vanskelig (målt med disse reduksjonene) som alle problemene i S .*

P, NP og NPC Dette er et tema som dekkes grundig ellers i pensum, men vi trenger noen helt grunnleggende definisjoner her. P , NP og NPC er *klasser av beslutningsproblemer*.

P er klassen av alle beslutningsproblemer som kan *løses* i polynomisk tid.

NP er klassen av alle beslutningsproblemer med en vitnerelasjon som kan *verifiseres* i polynomisk tid.[†]

Et beslutningsproblem er *NP-hardt* dersom det finnes Karp-reduksjoner til det *fra alle problemer i NP*.

De NP-harde problemene i NP kalles *NP-komplette*, og utgjør klassen NPC .

Med andre ord, om vi bruker Karp-reduksjoner som sammenligning, så er de NP-komplette problemene de vanskeligste i NP .

Vi kan også bruke disse klassene til å *beskrive søkeproblemer*.

Et *NP-søkeproblem* er søkeproblemet til en relasjon som kan verifiseres i polynomisk tid.[‡] Tilhørende beslutningsproblem er altså i klassen NP .

Vi sier at et søkeproblem er *NP-hardt* dersom det finnes Levin-reduksjoner til det *fra alle NP-søkeproblemer*.[‡]

Det er trivielt å redusere fra et beslutningsproblem i NP til et tilhørende NP-søkeproblem (med en Cook-reduksjon), så NP-harde søkeproblemer er minst like vanskelige som alle problemer i NP .

Reduksjon fra optimering til terskling

En av de vanligste problemtypene vi kommer borti er *optimeringsproblemer* (eller *optimaliseringsproblemer*). Dette er en type søkeproblem, der hver potensielle løsning har en eller annen verdi eller kostnad, og vi er ute etter å finne den beste (eller en av de beste, om det finnes flere slike). Disse problemene er gjerne *ikke* NP-søkeproblemer; det er som regel vanskelig å verifisere at en løsning *er best*.

Men: Ethvert slikt problem har et tilhørende *tersklingsproblem*, der vi er ute etter en løsning med verdi eller kostnad som tilfredsstillen en eller annen terskel, og disse problemene er ofte NP-søkeproblemer. For eksempel, heller enn å finne den korteste stien fra s til t , er vi ute etter en eller annen sti som ikke er lengre enn en terskel k . Det er forhåpentligvis ganske opplagt at vi kan redusere fra terskling til optimering, dvs., at vi kan løse tersklingsproblemet ved hjelp av optimeringsproblemet.

* Her er «hard» oversatt fra engelsk, i betydningen «hard problems», og burde nok helst vært oversatt til «vanskelig», men det er vanlig å bruke «hard» også på norsk.

† Her kreves også at sertifikatet har polynomisk størrelse, så kjøretiden til verifikasjonen blir polynomisk som funksjon av *instansens* størrelse.

‡ Teknisk sett er det da FNP-hardt (se side 29).

For å kunne utvide begrepet NP-hardhet til optimering, holder det ikke nødvendigvis med Levin-reduksjoner, siden de ikke har lov til å «kaste» løsninger, som diskutert på side 24. Optimeringsproblemer har gjerne *alltid* løsninger, og da kan vi ikke redusere fra et søkeproblem som av og til *ikke* har det.

En mulighet er å fokusere på tersklingsproblemet, og beskrive *det* som NP-hardt. En annen mulighet er å tillate generelle Cook-reduksjoner, som altså *kan* kaste løsninger, som i FIND-SHORT-PATH, nedenfor.

Her har vi et eksempel på en prosedyre for å finne en sti fra s til t i grafen G , der lengden på stien er maks k :

FIND-SHORT-PATH(G, s, t, k)

```
1 // TODO: Find shortest path  $p$  from  $s$  to  $t$ 
2 if the length of  $p$  is at most  $k$ 
3   return  $p$ 
4 else return NIL
```

Det som kanskje er mer interessant er at vi i dette tilfellet også kan gå motsatt vei! La oss si at lengden bare er antall kanter (dvs., at vi ikke har en *vektet* graf). Da kan den korteste stien fra s til t maksimalt ha $|G.V| - 1$ kanter,* så vi kan bare prøve oss frem:

FIND-SHORTEST-PATH(G, s, t)

```
1 for  $k = 0$  to  $|G.V| - 1$ 
2   // TODO: Find a path  $p$  with length  $\leq k$  from  $s$  to  $t$ 
3   if  $p \neq \text{NIL}$ 
4     return  $p$ 
```

Kjøretiden blir altså $\Theta(V)$ ganger tiden det tar å finne en k -sti. Her har vi flere kall til orakelet, som er helt i orden i Cook-reduksjoner generelt.

Siden vi i dette tilfellet har effektive reduksjoner begge veier mellom optimering og tersklings, har vi vist at de to problemtypene er like vanskelige. Det vil si, om vi har en algoritme med polynomisk kjøretid for det ene problemet, så gir det oss en algoritme med polynomisk kjøretid også for det andre. Eller motsatt, om vi *ikke* har en polynomisk algoritme for det ene problemet, så har vi heller ingen slik algoritme for det andre. De er altså *ekvivalente* (*computationally equivalent*) under polynomiske reduksjoner.

Men er det sant generelt? Hva om reduksjonen vår må bruke et veldig stort antall orakelspøringer? Vel, det kommer litt an på. Men: Dersom verdiene til de ulike løsningene kan representeres som et heltall som vi kan utføre operasjoner på i konstant tid, så antar maskinmodellen vår (RAM-modellen) at de ikke blir mer enn *eksponentielt store*, og i så fall kan vi bruke et slags *binærsøk* (se appendiks D) i verdiområdet for å finne optimum. For eksempel, dersom vi har en *vektet* graf, så stien p har lengde eller vekt $w(p)$, som er et ikke-negativt heltall:

FIND-SHORTEST-WEIGHTED-PATH(G, w, s, t)

```
1 let  $p$  be some path  $s \rightsquigarrow t$ 
2  $lo, hi = 0, w(p)$ 
3 while  $lo < hi$ 
4    $k = \lfloor (lo + hi) / 2 \rfloor$ 
5   // TODO: Find a path  $q$  with  $w(q) \leq k$ 
6   if  $q \neq \text{NIL}$ 
7      $p, hi = q, k$ 
8   else  $lo = k + 1$ 
9 return  $p$ 
```

* Her er altså $G.V$ nodemengden til grafen. Hvorfor vi får $|G.V| - 1$ blir forklart i kap. 22.1 i læreboka.

Antall «ja-nei»-spørsmål i et binærsøk, som altså her blir antall kall til orakelet, er *logaritmisk* som funksjon av verdiområdet. Dvs., om vi starter med maksverdi w , så vil vi ha $O(\lg w)$ iterasjoner. Dette tilsvarer antall bits vi trenger for å beskrive w , og er dermed polynomisk som funksjon av inputstørrelsen. (Se også appendiks E, for en diskusjon av dette, og begrepet *pseudopolynomisk*.)

Med andre ord: Om vi har et eksponentielt verdiområde (og ikke *mer* enn det), og vi kan bruke binærsøk med terskelen, så kan vi redusere mellom optimering og terskling.

Reduksjon fra søk til beslutning

Hvis vi kan løse et søkeproblem, kan vi naturligvis også løse det tilhørende beslutningsproblemet. Det vil si, om vi er i stand til å *finne* en løsning, dersom den finnes, så kan vi også svare på *om* det finnes en løsning. Men også *her* er det, for mange naturlige søkeproblemer, mulig å redusere motsatt vei. En strategi er å bygge en løsning gradvis, ved å prøve oss frem, og så bruke beslutnings-orakelet til å sjekke *om vi er på rett vei!*

La oss for eksempel si at vi vil finne en sti fra s til t i en rettet graf, og vi vil redusere det til å avgjøre om en slik sti eksisterer:

```

FIND-PATH( $G, s, t$ )
1  if  $s == t$ 
2      return  $\langle s \rangle$ 
3  for each vertex  $v$  in  $G.Adj[s]$ 
4      // TODO: Is there a path from  $v$  to  $t$  in  $G - s$ ?
5      if there is such a path
6           $p = \text{FIND-PATH}(G - s, v, t)$ 
7          return  $\langle s, p_1, \dots \rangle$ 
8  return NIL

```

Hvis en kant er første skritt i riktig retning (sjekket i linje 4), finner vi resten av løsningen rekursivt i linje 6. Reduksjonen har polynomisk kjøretid, så om vi kan *avgjøre* om det eksisterer en sti fra s til t i polynomisk tid, så kan vi også *finne* en slik sti i polynomisk tid.

Strategien vi følger her, mer generelt, er å redusere et søkeproblem til et *beslektet* beslutningsproblem, som sjekker om en uferdig løsning er «lovende», altså om det er mulig å bygge den ut til en fullstendig løsning. For eksempelet over, var dette bare beslutningsproblemet implisitt i søkeproblemet (dvs., «Finnes det en sti?»), men det er absolutt ikke opplagt om det alltid er mulig å gjøre det slik. Kanskje det å avgjøre om en løsning kan fullføres er et *litt annet* problem enn å avgjøre om det eksisterer en løsning?

La oss se på et viktig spesialtilfelle, der relasjonen kan verifiseres i polynomisk tid, og det tilhørende beslutningsproblemet er NP-komplett.*

1. Det finnes en polynomisk algoritme som kan svare på spørsmålet «Er dette en løsning?»
2. Vi kan bruke samme algoritme til å svare på spørsmålet «Kan denne partielle løsningen fullføres på denne måten?»
3. Det betyr at spørsmålet «Kan denne partielle løsningen fullføres?» er i NP.
4. Det betyr at spørsmålet «Kan denne partielle løsningen fullføres?» må kunne reduseres til spørsmålet «Finnes det en løsning?», siden sistnevnte er NP-komplett.
5. Vi vet at spørsmålet «Finn en løsning» kan reduseres til «Kan denne partielle løsningen fullføres?», som altså kan reduseres videre til «Finnes det en løsning?».

* Relasjonen vil da altså være en av de mulige vitnerelasjonene som viser at beslutningsproblemet er i NP.

Akkurat hvordan vi bygger partielle løsninger er fleksibelt, men helt generelt kan vi bygge løsningen én bit om gangen: «Har vi en løsning som begynner på 0?», etc., for et gradvis økende bitstreng-prefiks.

Med andre ord: For et NP-komplett problem med tilhørende vitnerelasjon* kan søkeproblemet reduseres til beslutningsproblemet i polynomisk tid!

Klasser av søkeproblemer

På samme måte som for beslutningsproblemer, så skiller vi mellom søkeproblemer vi kan *løse* i polynomisk tid, og problemer der vi kan *sjekke* en løsning i polynomisk tid. Vi snakker da gjerne f.eks. om et «søkeproblem for et (beslutnings-)problem i NP», eller bare et «NP-søkeproblem».

Om vi starter med formalismen med beslutningsproblemer, handler dette altså om å lete etter et *bevis* for at svaret på et beslutningsproblem i NP er «ja». Det er akkurat den typen bevis som kan verifiseres i polynomisk tid.

Det betyr ikke at søkeproblemet *er i klassen NP*, siden den klassen kun består av beslutningsproblemer, men litt uformelt sier vi gjerne likevel at problemet «er i NP». Det vi da mener, mer formelt, er at løsninger på problemet kan verifiseres i polynomisk tid, som betyr at det tilhørende beslutningsproblemet ligger i NP. På samme måte er det ikke uvanlig at man sier at et søkeproblem «er i P» dersom det lar seg løse i polynomisk tid.

Merk: Selv om beslutningsproblemet er i P, *vet vi ikke nødvendigvis* om søkeproblemet kan løses i polynomisk tid! For eksempel, det å avgjøre om et heltall kan faktoriseres er i P,[†] mens de fleste tror at det å faktisk *faktorisere* ikke kan gjøres i polynomisk tid.[‡]

Og: Samme beslutningsproblem kan tilhøre flere relasjoner. Selv om problemet er i NP, er det ikke gitt at *alle* matchende relasjoner kan verifiseres i polynomisk tid!

Dersom et søkeproblem kan verifiseres i polynomisk tid, og det er NP-hardt (under Levin-reduksjoner), så hender det på samme måte at vi (uformelt) sier at det er NP-komplett, selv om det egentlig er det tilhørende beslutningsproblemet som er det.

For spesielt interesserte: Søkeproblemer som kan løses i polynomisk tid, tilhører klassen FP; de som kan verifiseres i polynomisk tid tilhører FNP, mens de NP-harde problemene (Levin) i FNP er FNP-komplette. Så med et «NP-komplett søkeproblem» mener vi egentlig et FNP-komplett problem. Siden den ene retningen av en Levin-reduksjon er en Karp-reduksjon, vil de tilhørende beslutningsproblemene være NP-komplette.

* Som altså kan verifiseres i polynomisk tid, og der sertifikatene er av polynomisk størrelse.

[†] Altså i to faktorer større enn 1. Se <https://annals.math.princeton.edu/2004/160-2/p12>.

[‡] Det er likevel lite trolig at faktorisering er NP-hardt (under Levin-reduksjoner), siden det ville bety at beslutningsproblemet er NP-komplett. Men siden det kan løses i polynomisk tid, ville det igjen bety at $P = NP$, som de fleste tror ikke er tilfelle.

Appendiks C

Iterasjonsmetoden

I læreboka er *rekursjonstrær* en sentral metode for å løse rekurrensligninger. Dette er en mer visuell fremgangsmåte enn den som ble brukt i tidligere utgaver, som de kalte *iterasjonsmetoden*. Det er egentlig akkurat samme prinsipp, bare at man jobber mer direkte på ligningene. Ideen er at man bruker rekurrensen selv til å ekspandere de rekursive termene. For eksempel:

$$\begin{aligned}T(0) &= 0 \\T(n) &= T(n-1) + 1 \quad (n \geq 1)\end{aligned}$$

Her ønsker vi å bli kvitt $T(n-1)$ på høyre side i ligningen. Om vi antar at $n > 1$, så kan vi bruke definisjonen $T(n) = T(n-1) + 1$ til å finne ut hva $T(n-1)$ er, nemlig

$$(T((n-1)-1) + 1) + 1$$

eller $T(n-2) + 2$. Det er altså én *ekspansjon* (eller iterasjon). Vi kan fortsette på samme måte, trinn for trinn, helt til vi ser et mønster dukke opp:

$$\begin{aligned}T(0) &= 0 \\T(n) &= T(n-1) + 1 & (1) \\&= T(n-2) + 2 & (2) \\&= T(n-3) + 3 & (3) \\&\vdots & \vdots \\&= T(n-i) + i & (i) \\&= T(n-n) + n = n & (n)\end{aligned}$$

Først ekspanderer vi altså $T(n)$, så $T(n-1)$ og så $T(n-2)$, og så videre, hele tiden ved hjelp av den opprinnelige rekurrensen. Linjenummeret (til høyre) viser hvor mange ekspansjoner vi har gjort. Etter hvert klarer vi altså å uttrykke resultatet etter et vilkårlig antall ekspansjoner, i . Her kan i være 1, 2, 3, eller et hvilket som helst annet tall, så lenge vi ikke ekspanderer oss «forbi» grunntilfellet $T(0) = 0$. Og det er nettopp grunntilfellet vi ønsker å nå; vi vil gi T argumentet 0. For å få til det må vi få $n-i$ til å bli 0, og altså sette $i = n$. Dette gir oss svaret, som vist i siste linje. Et litt mer interessant eksempel finner du i appendiks D, der hver ekspansjon *halverer* argumentet til T , så vi bare trenger lg n ekspansjoner før vi kommer ned til $T(1)$, som er grunntilfellet der.

Appendiks D

Binærsøk

Læreboka sin beskrivelse av binærsøk, på engelsk *binary search* eller *bisection*, begrenser seg til et par oppgaver (2.3-6 og 4.5-3, med problemet definert i 2.1-4). Algoritmen er gjennomgått i mer detalj i forelesningene, men den sentrale informasjonen er oppsummert her. Selve algoritmen er definert som følger, der tabellen $A[1:n]$ antas å være sortert.

```
BISECT(A, p, r, v)
1  if  $p \leq r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      if  $v == A[q]$ 
4          return  $q$ 
5      else if  $v < A[q]$ 
6          return BISECT(A, p, q - 1, v)
7      else return BISECT(A, q + 1, r, v)
8  else return NIL
```

Dersom tabellen A inneholder v , som er verdien vi leter etter, så vil $\text{BISECT}(A, 1, n, v)$ returnere en indeks i som er slik at $A[i] = v$. Hvis ikke, så returneres NIL.

Korrekthet kan vises ved induksjon over lengden til segmentet $A[p:r]$. Vi har to grunntilfeller: Enten er $p > r$, så segmentet er tomt, og vi returnerer NIL, ellers så har vi $p = r$ og $v = A[q]$, og vi returnerer q . Begge disse er korrekte. Anta så at $p < r$, og at BISECT gir rett svar for kortere intervaller. Både $A[p:q-1]$ og $A[q+1:r]$ er kortere intervaller, så samme hvilket rekursivt kall som velges, så vil vi returnere rett indeks, siden vi leter i rett halvdel (vha. betingelsen i linje 5).

For å finne kjøretiden kan vi for eksempel telle antall ganger sammenligningen $p \leq r$ utføres. Vi kan gjøre noen antagelser for å forenkle utregningen. Disse har ingenting å si for den asymptotiske kjøretiden. For det første antar vi at $n = 2^k$, for et eller annet heltall $k \geq 0$, slik at vi alltid kan dele nøyaktig på midten, helt til vi ender med $p == q$. For det andre så antar vi at v finnes i A , så rekursjonen stanser her. For å sikre at vi deler på midten, kan vi til og med anta at den eneste forekomsten av v er $A[n]$, så vi alltid velger det høyre segmentet, $A[q+1:r]$. (Vi kan bruke induksjon, dvs., *substitusjonsmetoden*, til å verifisere at den asymptotiske kjøretiden vi finner er gyldig også om disse antagelsene ikke gjelder.) For eksempel, hvis $n = 4$, $p = 1$ og $r = 4$, så blir $q = \lfloor (1+4)/2 \rfloor = 2$, og intervallet $q+1 \dots, r$ får 2 = $n/2$ elementer. Vi får følgende rekurrensutregning.

$$\begin{aligned} T(1) &= 1 \\ T(n) &= T(n/2) + 1 & (1) \\ &= T(n/4) + 2 & (2) \\ &= T(n/8) + 3 & (3) \\ &\vdots & \vdots \\ &= T(n/2^i) + i & (i) \\ &= T(n/n) + \lg n = \lg n + 1 & (\lg n) \end{aligned}$$

Med andre ord er kjøretiden $\Theta(\lg n)$.

Som så ofte ellers, så hjelper det å tenke rekursivt for å forstå hvordan algoritmen fungerer, men det er, som alltid, mulig å kvitte seg med rekursjonen (se Chapter G) og få en iterativ versjon:

```

BISECT'(A, p, r, v)
1  while  $p \leq r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      if  $v == A[q]$ 
4          return  $q$ 
5      else if  $v < A[q]$ 
6           $r = q - 1$ 
7      else  $p = q + 1$ 
8  return NIL

```

Denne versjonen vil generelt være mer effektiv, siden man slipper ekstra kostnader ved funksjonskall.* Både korrekthetsbevis og kjøretidsberegning kan oversettes ganske direkte fra BISECT til BISECT'.

* Siden returverdien fra det rekursive kallet returneres direkte, uten noe mer kode imellom – et såkalt *tail call* – så vil enkelte språk kunne gjøre denne transformasjonen automatisk, ved hjelp av såkalt *tail-call elimination* eller *tail-call optimization*.

Appendiks E

Ryggsekkproblemet

Det såkalte *ryggsekkproblemet* kommer i flere varianter, og to av dem er beskrevet på side 428–430 i læreboka. Den *kontinuerlige* varianten er lett å løse: Man tar bare med seg så mye som mulig av den dyreste gjenstanden, og fortsetter nedover på lista, sortert etter kilopris. I den binære varianten derimot, blir ting litt vanskeligere—her må man ta med en hel gjenstand eller la den ligge. Løsningen er egentlig beskrevet på 429, men den er litt bortgjemt i teksten, og beskrivelsen er svært skissepreget.

Først kan vi definere problemet litt ordentlig:

Input: To sekvenser med n positive heltall, *verdier* $\langle v_1, \dots, v_n \rangle$ og *vekter* $\langle w_1, \dots, w_n \rangle$, samt et positivt heltall, *kapasiteten* W .

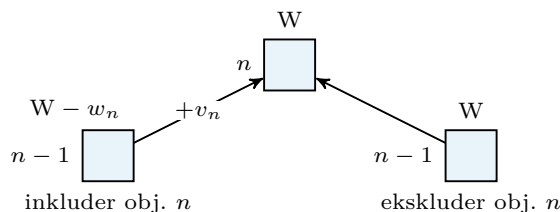
Output: En mengde gjenstander i der totalverdien (summen av v_i) er størst mulig, mens totalvekten (summen av w_i) er maksimalt W .

For enkelhets skyld fokuserer vi gjerne bare på å finne totalverdien. Å utvide til å finne hvilke gjenstander som inngår, krever bare at vi lagrer hvilke valg vi gjør underveis (som f.eks. med pilene i LCS-LENGTH, s. 397 i boka).

Akkurat som i for eksempel FLOYD-WARSHALL så baserer dekomponeringen seg på et *ja-nei*-spørsmål. I dette tilfellet: «Skal vi ta med gjenstand i ?» For hver av de to mulighetene sitter vi igjen med en delinstans som vi løser rekursivt (i hvert fall konseptuelt). Som vanlig tenker vi oss at dette er siste trinn, og antar at vi har gjenstander $1, \dots, i$ tilgjengelige. Vi har da følgende alternativer:

- (i) Ja, vi tar med gjenstand i . Vi løser så problemet for gjenstander $1, \dots, i-1$ men der kapasiteten er redusert med w_i . Vi legger så til v_i til slutt.
- (ii) Nei, vi tar ikke med gjenstand i . Vi løser problemet for gjenstander $1, \dots, i-1$, men kan fortsatt bruke hele kapasiteten. Til gjengjeld får vi ikke legge til v_i .

Situasjonen er illustrert i figur E.1, der hver rute representerer en delløsning (en celle i løsningstabellen, for eksempel) og pilene er avhengigheter, som vanlig. Her er W den totale kapasiteten, og n er antall gjenstander. Vi kan sette opp en rekursiv løsning slik (der vi antar at vektene og verdiene er globale variable, for enkelhets skyld):



Figur E.1: Dekomponering for det binære ryggsekkproblemet.

```

KNAPSACK( $n, W$ )
1  if  $n == 0$ 
2      return 0
3   $x = \text{KNAPSACK}(n - 1, W)$ 
4  if  $w_n > W$ 
5      return  $x$ 
6  else  $y = \text{KNAPSACK}(n - 1, W - w_n) + v_n$ 
7      return  $\max(x, y)$ 

```

Denne prosedyren vil naturligvis ha eksponentiell kjøretid. Vi kan løse det ved å bruke memoisering, eller vi kan skrive den om til en iterativ *bottom-up*-løsning, som nedenfor. Her er noen linjer gjort mindre fremtredende, for å fremheve likheten mellom de to prosedyrene, der tabellen K i $\text{KNAPSACK}'$ på mange vis har samme rolle som selve den rekursive prosedyren KNAPSACK .

```

KNAPSACK'( $n, W$ )
1  let  $K[0:n, 0:W]$  be a new table
2  for  $j = 0$  to  $W$ 
3       $K[0, j] = 0$ 
4  for  $i = 1$  to  $n$ 
5      for  $j = 0$  to  $W$ 
6           $x = K[i - 1, j]$ 
7          if  $j < w_i$ 
8               $K[i, j] = x$ 
9          else  $y = K[i - 1, j - w_i] + v_i$ 
10              $K[i, j] = \max(x, y)$ 
11 return  $K[n, W]$ 

```

Forskjellen ligger i at vi her eksplisitt legger inn **for**-løkker for å gå gjennom alle delinstansene, heller enn å gjøre det direkte (implisitt) med rekursjon.

Men dette er ikke polynomisk!

Det binære ryggsekkproblemet er et såkalt NP-hardt problem, og det er ingen som har funnet noen polynomisk løsning på det; trolig vil ingen noen gang finne det heller. Det ser jo ut til å motsi diskusjonen over, der vi kom frem til det som absolutt *ser ut som* en algoritme med polynomisk kjøretid. Hva er det som foregår her?

Kjøretiden til både KNAPSACK og $\text{KNAPSACK}'$ er $\Theta(nW)$, siden det er nW delinstanser, og vi utfører en konstant mengde arbeid per delinstans. Og det er rett at dette er en polynomisk funksjon av n og W , men det er ikke nok til at vi kan si at algoritmene «har polynomisk kjøretid». Spørsmålet er *som funksjon av hva?* Hvis vi eksplisitt sier «som funksjon av n og W » er alt i orden. Men om vi ikke oppgir noen eksplisitte parametre, så antas kjøretiden å være en funksjon *av input-størrelsen*, som altså er hvor stor plass en instans tar. Akkurat hvordan vi måler denne størrelsen kan avhenge av problemet (se side 28 i læreboka), men når vi regner på om ting kan løses i polynomisk tid (i forbindelse med NP-kompletthet og denslags) holder vi oss til *antall bits* i input, i en rimelig encoding. Størrelsen blir da $\Theta(n \lg n + \lg W)$, siden vi bare trenger $\Theta(\lg W)$ bits for å lagre parameteren W . Poenget er altså at W vokser eksponentielt som funksjon av $\lg W$, og kjøretiden er, teknisk sett, eksponentiell.

Dette er kanskje enda enklere å se hvis vi lar m være antall bits i W , så vi kan skrive kjøretiden som

$$T(n, m) = \Theta(n2^m).$$

Da er det forhåpentligvis tydelig at dette ikke er en polynomisk kjøretid. Kjøretider som er polynomiske hvis vi lar et *tall fra input* være med som parameter til kjøretiden (slik som $\Theta(nW)$, der W er et tall fra input, og ikke direkte en del av problemstørrelsen) kaller vi *pseudopolynomiske*.

RAM-modellen og pseudopolynomialitet

Historien er egentlig en overforenkling, som utelater et viktig poeng. Om vi for enkelhets skyld sier at problemstørrelsen er n , så krever RAM-modellen (s. 26–27) at $m \leq c \lg n$, for en eller annen konstant $c \geq 1$, hvis vi skal kunne bruke W til indeksering og aritmetikk i konstant tid.

Siden $m \leq c \lg n$ og $W \leq 2^m$, får vi $W \leq 2^{c \lg n} = (2^{\lg n})^c = n^c$ og dermed at

$$\Theta(nW) = \Theta(n^{c+1}),$$

som betyr at kjøretiden plutselig er polynomisk likevel!

Saken er at vi også er interesserte i dette problemet for enda større verdier av W , først og fremst fordi vi da kan vise at det er NP-hardt. Da gir ikke KNAPSACK (eller KNAPSACK') lenger noen løsning i polynomisk tid. Det gjelder selv om vi utvider dem til å faktisk kunne håndtere større heltall, fordi operasjonene på W da tar *mer enn konstant tid*, og kjøretiden da bare blir *mer enn* $\Theta(nW)$.

Appendiks F

Generell grafteraversering

Læreboka beskriver to traverseringsalgoritmer, *bredde-først-søk* (20.2) og *dybde-først-søk* (20.3). Disse fremstilles ganske ulikt, men er nært beslektede. Prosedyren BFS (på side 556) kan tilpasses til å oppføre seg omtrent likt med DFS-VISIT (side 565), ved å bytte ut FIFO-køen Q med en LIFO-kø, eller stakk (*stack*).

Merk at prosedyren DFS, slik den er beskrevet i boka, ikke har noen startnode. Den starter bare fra hver node etter tur, til den har nådd hele grafen. Begrepet «dybde-først-søk» er noe tvetydig, og brukes både om atferden til DFS og DFS-VISIT.

Køen Q i BFS er rett og slett en liste med noder vi har oppdaget via kanter fra tidligere besøkte noder, men som vi ennå ikke har besøkt. Noder er hvite før de legges inn, grå når de er i Q og svarte etterpå. Det at vi bruker en FIFO-kø er det som lar BFS finne de korteste stiene til alle noder, siden vi utforsker grafen «lagvis» utover, men vi kan egentlig velge vilkårlige noder fra Q i hver iterasjon, og vi vil likevel traversere hele den delen av grafen vi kan nå fra startnoden.

Grunnen til at en LIFO-kø gir oss samme atferd som en rekursiv traversering (som i DFS-VISIT) er at vi egentlig bare simulerer hvordan rekursjon er implementert. Internt bruker maskinen en *kallstakk*, der informasjon om hvert kall legges øverst, og hentes frem igjen når rekursive kall er ferdige.

Som Cormen mfl. påpeker, så bruker Prims og Dijkstras algoritmer svært lignende ideer. Prosedyrene MST-PRIM (side 596) og DIJKSTRA (side 620) forenkler ting litt ved å sette alle noder fra G.V inn i Q før løkken som bruker Q, så vi mister den delen av traverseringen som handler om å *oppdage* noder. I eldre fremstillinger av disse algoritmene så ligner de mer på BFS, og noder legges inn i Q når de oppdages. Den sentrale forskjellen er hvilke noder som tas *ut* av Q, altså hvilke noder som *prioriteres*.

- I BFS prioriteres de eldste og i DFS-VISIT prioriteres de nyeste;
- I DIJKSTRA prioriteres noder med lavt avstandsestimat ($v.d$), mens
- I MST-PRIM prioriteres noder som har en lett kant til én av de besøkte (svarte) nodene.

Prioritetene oppdateres underveis, som nødvendig. Andre prioriteringer vil gi andre traverseringsalgoritmer, som for eksempel den såkalte A*-algoritmen (ikke pensum).

Appendiks G

Rekursjon og iterasjon

Iterative funksjoner kan oversettes til ekvivalente rekursive funksjoner, og omvendt. Dette er noe vi ofte gjør under konstruksjon av algoritmer basert på rekursiv dekomponering, kanskje spesielt innen dynamisk programmering. De to måtene å tenke på kan være anvendelige i ulike situasjoner; for eksempel kan det være praktisk å tenke rekursivt når man bygger løsninger fra del-løsninger, men iterative løsninger unngår ekstrakostnader til funksjonskall, som kan være viktig når en algoritme faktisk skal implementeres.

Fra iterasjon til rekursjon

Dette er en overgang som ofte dukker opp om man skal lære seg såkalt funksjonell programmering. Å leke seg med funksjonelle språk kan være nyttig for å bli grundig vant til rekursjon.*

Det enkleste å oversette direkte er kanskje løkker av typen *repeat-while* (i mange språk kjent som *do-while*), der betingelsen kommer til slutt. For eksempel:

```
1 repeat
2   body
3 while condition
```

Dette kan man oversette til:

```
FUNC(vars)
1 body
2 if condition
3   FUNC(vars)
```

Ett kall tilsvare her én iterasjon av løkken, og *vars* tilsvare lokale variable som sendes med fra iterasjon til iterasjon. La oss si vi f.eks. vil skrive ut tallene $1 \dots n$.

```
1 i = 1
2 repeat
3   print i
4   i = i + 1
5 while i ≤ n
```

Vi kan skrive om det til:

```
NUMBERS(i, n)
1 print i
2 i = i + 1
3 if i ≤ n
4   NUMBERS(i, n)
```

Funksjonen kalles med argumenter 1 og *n* til å begynne med.

Intuitivt kan dette tolkes på flere måter. En veldig enkel tolkning er at kallet til NUMBERS forteller maskina at vi skal gå tilbake til start, og utføre definisjonen av NUMBERS på nytt,

* Ta for eksempel en titt på Haskell (<https://haskell.org>).

akkurat som i en løkke. Det blir som om vi hadde skrevet om løkken til det følgende (om vi antar at i alt er satt til 1):

```
1 print  $i$ 
2  $i = i + 1$ 
3 if  $i \leq n$ 
4     go to 1
```

Internt er det nettopp denne typen *go to*-utsagn kompilatoren oversetter løkker til. I mange språk er det også nettopp slik kode man får om man bruker rekursjon slik som vi har gjort over! Med andre ord: Både løkker og rekursjon kan ende med samme primitive maskinkode, som inneholder verken løkker eller rekursjon.

En annen måte å forstå koden på, intuitivt, er at vi skiller mellom første iterasjon og *resten*. Første iterasjon består av linje 1 og 2, og om vi ikke har kommet til slutten, så utfører vi resten av iterasjonene rekursivt.

Om man trenger verdier som beregnes i løkken, kan disse returneres fra funksjonen. For eksempel kan returverdiene være de samme variablene som før:

```
FUNC( $vars$ )
1 body
2 if condition
3      $vars = \text{FUNC}(vars)$ 
4 return  $vars$ 
```

Merk at nå kan ikke lenger kompilatoren uten videre oversette dette til et enkelt *go to*-utsagn (såkalt *tail-call optimization*) siden vi har kode etter det rekursive kallet.

La oss si at vi f.eks. vil *summere* tallene fra 1 til n , i stedet, med en løkke som:

```
1  $s, i = 0, 1$ 
2 repeat
3      $s = s + i$ 
4      $i = i + 1$ 
5 while  $i \leq n$ 
```

Vi kan skrive om dette til:

```
SUM( $i, n, s$ )
1  $s = s + i$ 
2  $i = i + 1$ 
3 if  $i \leq n$ 
4      $i, n, s = \text{SUM}(i, n, s)$ 
5 return  $i, n, s$ 
```

Kallet som starter det hele blir SUM(1, n , 0). (Her er det naturligvis unødvendig å sende med *alle* variablene tilbake, siden vi kun trenger s , men det illustrerer at oversettelsen kan gjøres ganske mekanisk.)

Å håndtere andre typer løkker er relativt rett frem. For eksempel kan du begynne med å skrive dem om til å være på *repeat-while*-formen (selv om du nok ofte vil kunne finne mer naturlige rekursive måter å skrive koden på).

Fra rekursjon til iterasjon

Som nevnt over: Hvis det rekursive kallet kommer helt til slutt i funksjonen, kan vi ofte oversette det direkte til en ekvivalent løkke. Dersom koden fortsetter etter et rekursivt kall, derimot, så må vi simulere rekursjonen på noe vis; vi må huske hvilke verdier de lokale variablene hadde *før* det rekursive kallet, og gi dem disse verdiene igjen etterpå!

Måten vi gjør dette på er med en stakk, akkurat slik som rekursjon er implementert internt i programmeringsspråkene (vha. en *kallstakk*). La oss si vi har en rekursiv funksjon som dette:

```
FUNC(...)  
1  before  
2  if condition  
3      FUNC(...)  
4  after
```

La oss også si at koden både før og etter bruker de samme variablene, *vars*. Da kan vi fjerne rekursjonen slik, der S er en LIFO-stakk:

```
1  repeat  
2      before  
3      PUSH(S, vars)  
4  until not condition  
5  while S is not empty  
6      vars = POP(S)  
7      after
```

Det rekursive kallet til FUNC mottar formodentlig andre argumenter enn dem vi får inn; det kan her bare håndteres med vanlige tilordninger.

Bibliografi

- [1] T. H. Cormen mfl. *Introduction to Algorithms*. 3. utg. The MIT Press, 2009.
- [2] T. H. Cormen mfl. *Introduction to Algorithms*. 4. utg. The MIT Press, 2022.
- [3] O. Goldreich. *P, NP, and NP-Completeness: The Basics of Computational Complexity*. Cambridge University Press, 2010.
- [4] B. A. Oakley. *A mind for numbers: How to excel at math and science (even if you flunked algebra)*. TarcherPerigee, 2014.